Day 81 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 12, Day 5 reference

Watch: Illustrated Guide to Transformers Neural Network (The AI Hacker) | Attention is all you need (Transformer): Model explanation (Umar Jamil)

`day81_transformer_forward.py`, segment by segment, in the order `main()` uses things. The attention functions, the encoding, LayerNorm, the FFN, Day 78's encoder block, Day 80's decoder block and Day 69's data code are copied verbatim and explained on those days. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: m = examples · n, T = source and decoder lengths · D = 32 numbers per token · V = 10 tokens (EOS, digits 1-8, GO) · E = the shared embedding table · PE = Day 77's encoding

Segment 0 — Setup

Settings

CODE

```
EOS, GO = 0, 9
V = 10
SEEDS = [0, 1, 2]
D, F, HEADS = 32, 64, 4
MAX_LEN = 128
```

- `EOS, GO = 0, 9` — Day 69's two special tokens, unpacked from one tuple.
- `D, F, HEADS = 32, 64, 4` — a small model: the task has only 10 tokens.

Segment 1 — Data

as_ids: from Day 69's one-hot rows to token ids

CODE

```
def as_ids(batch):
    Xs, Xd, Y, src = batch
    return src, Xd.argmax(axis=-1), Y
```

- `Xs, Xd, Y, src = batch` — unpacks the four arrays Day 69's make_batch returns.
- `Xd.argmax(axis=-1)` — the position of the 1 in each one-hot row is the token id.

Segment 2 — The model

head_mask and Transformer.__init__

CODE

```
def head_mask(mask, h):
    return np.repeat(mask[:, None, None, :], h, axis=1).reshape(mask.shape[0] * h, 1,
        mask.shape[1])
class Transformer:
    def __init__(self, n_enc=1, n_dec=1, seed=0, embed_scale=1.0, out_scale=1.0):
        rng = np.random.RandomState(seed)
        self.P = dict(E=rng.randn(V, D) * embed_scale)
        self.enc = [EncoderBlock(rng) for _ in range(n_enc)]
        self.dec = [DecoderBlock(rng, causal=True, cross=True) for _ in range(n_dec)]
        self.P.update(Wout=rng.randn(D, V) / np.sqrt(D) * out_scale, bout=np.zeros(V))
        self.PE = sinusoidal_encoding(MAX_LEN, D)
```

- `mask[:, None, None, :]` — (m, n) becomes (m, 1, 1, n), ready to repeat once per head.
- `[EncoderBlock(rng) for _ in range(n_enc)]` — a list of blocks sharing one rng, so each gets different weights.
- `rng.randn(D, V) / np.sqrt(D) * out_scale` — out_scale = 0.1 is Section 4's smaller output layer.

params, encode, decode, forward, loss

MATH

$$\log p = z - \max z - \log \Sigma \exp(z - \max z), \quad z = \text{dec } W_{out} + b_{out}$$

CODE

```
def params(self):
    out = dict(self.P)
    for i, b in enumerate(self.enc):
        out.update({f"e{i}.{k}": v for k, v in b.P.items()})
    for i, b in enumerate(self.dec):
        out.update({f"d{i}.{k}": v for k, v in b.P.items()})
    return out
def encode(self, src, smask):
    x = self.P["E"][src] + self.PE[:src.shape[1]]
    km = head_mask(smask, HEADS)
    for b in self.enc:
        x = b.forward(x, 0.0, km)
    return x
def decode(self, tgt_in, tmask, enc, smask):
    y = self.P["E"][tgt_in] + self.PE[:tgt_in.shape[1]]
    for b in self.dec:
        y = b.forward(y, 0.0, tmask, enc, smask)
    return y
def forward(self, src, smask, tgt_in, tmask):
    self.enc_out = self.encode(src, smask)
    self.dec_out = self.decode(tgt_in, tmask, self.enc_out, smask)
    logits = self.dec_out.reshape(-1, D) @ self.P["Wout"] + self.P["bout"]
    logp = logits - logits.max(axis=1, keepdims=True)
    return logp - np.log(np.exp(logp).sum(axis=1, keepdims=True))
def loss(self, src, smask, tgt_in, tmask, y):
    logp = self.forward(src, smask, tgt_in, tmask)[np.arange(y.size), y.ravel()]
    return -np.sum(logp * tmask.ravel()) / tmask.sum()
```

- `{f"e{i}.{k}": v for k, v in b.P.items()}` — names such as 'e0.Wq': block 0 of the encoder, its Wq.
- `self.P["E"][src] + self.PE[:src.shape[1]]` — fancy indexing gives (m, n, D); the (n, D) table broadcasts over the batch.

- `b.forward(x, 0.0, km)` — the positions were added once at the input, so the block's own pe argument is 0.0.
- `self.dec_out.reshape(-1, D)` — -1 lets NumPy work out m * T.

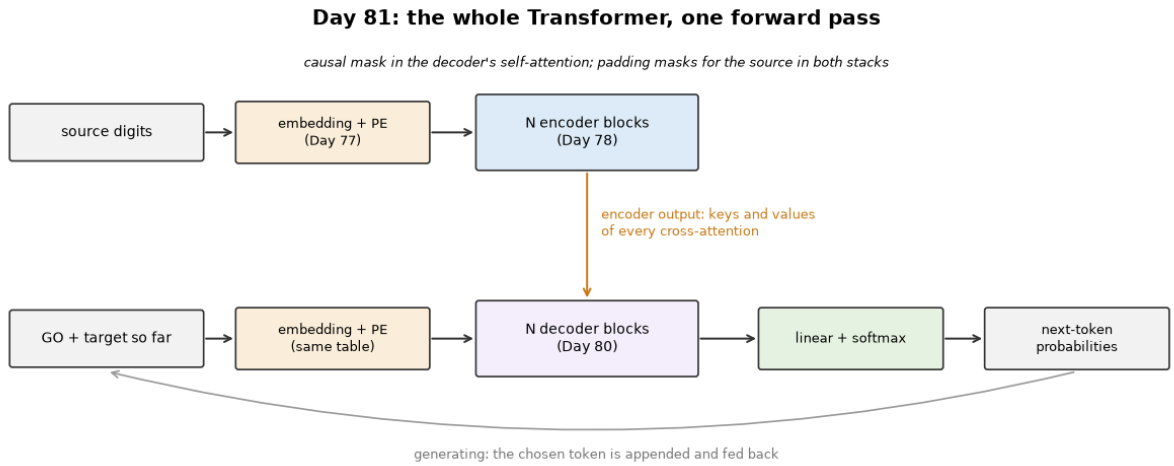
generate: one token at a time

CODE

```
def generate(self, src, n_steps):
    m = len(src); smask = np.ones(src.shape, bool)
    enc = self.encode(src, smask)
    tokens = np.full((m, 1), GO)
    steps = []
    for _ in range(n_steps):
        y = self.decode(tokens, np.ones(tokens.shape, bool), enc, smask)
        logits = y[:, -1] @ self.P["Wout"] + self.P["bout"]
        lp = logits - logits.max(axis=1, keepdims=True); lp -= np.log(np.exp(lp).sum(axis=1,
            keepdims=True))
        steps.append(lp)
        tokens = np.concatenate([tokens, lp.argmax(axis=1)[:, None]], axis=1)
    return tokens[:, 1:], np.stack(steps, axis=1)
```

- `np.full((m, 1), GO)` — an (m, 1) array of GO tokens: every output starts there.
- `y[:, -1] @ self.P["Wout"]` — only the last position's scores are needed for the next token.
- `np.concatenate([tokens, ...[:, None]], axis=1)` — appends one column: the chosen token.
- `np.stack(steps, axis=1)` — a list of (m, V) arrays becomes (m, n_steps, V).

SYSTEM DIAGRAM



The whole Transformer, one forward pass.

Segment 3 — Experiments

demo_params: summing weights by name

CODE

```
part = lambda pre: sum(v.size for k, v in P.items() if k.startswith(pre))
emb, out = P["E"].size, P["Wout"].size + P["bout"].size
enc, dec = part("e"), part("d")
```

- `k.startswith(pre)` — True for every name beginning with pre, e.g. 'e0.Wq' for 'e'.
- `sum(v.size for ...)` — a generator expression inside sum(): no list is built.

demo_masks: three changes

CODE

```
sm = np.ones(src.shape, bool); sm[:, 6:] = False
tm = np.ones(tgt_in.shape, bool)
base = net.forward(src, sm, tgt_in, tm).reshape(4, 9, V)
s2 = src.copy(); s2[:, 6:] = np.random.RandomState(3).randint(1, 9, size=(4, 2))
a = np.abs(net.forward(s2, sm, tgt_in, tm).reshape(4, 9, V) - base).max()
t2 = tgt_in.copy(); t2[:, 5] = (t2[:, 5] % 8) + 1
d = np.abs(net.forward(src, sm, t2, tm).reshape(4, 9, V) - base)
s3 = src.copy(); s3[:, 2] = (s3[:, 2] % 8) + 1
c = np.abs(net.forward(s3, sm, tgt_in, tm).reshape(4, 9, V) - base)
```

- `(t2[:, 5] % 8) + 1` — a different digit for sure: % 8 then + 1 maps 1..8 to 2..8, 1.
- `d[:, :5].max()` — the largest change at positions 0..4, before the changed input.
- `c.max(axis=2).min()` — the biggest change per position, then the smallest of those: every position moved at least this much.

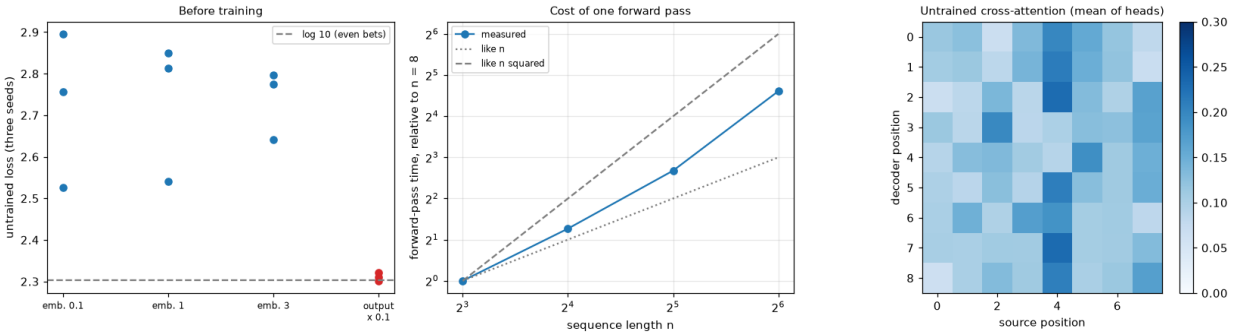
demo_decoding: decoding against one pass

CODE

```
tokens, steps = net.generate(src, 9)
fed = np.concatenate([np.full((300, 1), GO), tokens[:, :8]], axis=1)
one = net.forward(src, np.ones(src.shape, bool), fed, np.ones(fed.shape, bool)).reshape(300, 9, V)
print(f"\nlargest difference between the two: {np.abs(one - steps).max():.2e}; same tokens chosen:
    {(one.argmax(axis=2) == tokens).all()}")
```

- `tokens[:, :8]` — the first 8 written tokens: with GO in front they are exactly what the decoder saw at each step.
- `(one.argmax(axis=2) == tokens).all()` — True only if every one of the 300 x 9 choices agrees.

REAL GRAPH



Real graph from the script: untrained loss, forward-pass cost, untrained cross-attention.

Quick Syntax Glossary — new or reinforced constructs

- `a[:, None, None, :]` — insert axes of size 1 for broadcasting.
- `reshape(-1, D)` — let NumPy compute one dimension.
- `np.full((m, 1), GO)` — an array filled with one value.
- `np.stack(list, axis=1)` — join equal-shaped arrays along a new axis.
- `k.startswith(pre)` — string prefix test.
- `sum(gen)` — sum a generator expression.
- `x % 8 + 1` — wrap a digit around 1..8.

End of Day 81 syntax reference. For the experiments and what they showed, see the Day 81 complete guide.